

Pertemuan 4

Exception, Debugging, Testing, Terminasi, dan Rekursi

Pemrograman untuk Ilmu Data
dan Komputasi

MA25-21008 Kelas RA dan RB

Rifky Fauzi

Triyana Muliawati, S.Si., M.Si.

Program Studi Matematika
Institut Teknologi Sumatera
Semester Ganjil 2026/2027

Materi pertemuan

- Program yang gagal dan program yang tidak berhenti.
- Exception handling, debugging, dan testing.
- Terminasi pada loop dan rekursi.
- Rekursi dari masalah matematika.
- Membaca kode salah dan memperbaiki alasan logikanya.

Masalah pertama: program jalan terus

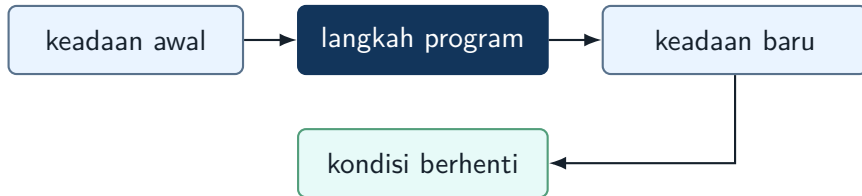
```
n = 5
while n > 0:
    print(n)
    n = n + 1
```

Pertanyaan awal

- Mengapa program tidak berhenti?
- Variabel mana yang berubah?
- Perubahan itu mendekati atau menjauhi kondisi berhenti?

Program yang benar bukan hanya menghasilkan output, tetapi juga memiliki alasan mengapa prosesnya berhenti.

Apa yang seharusnya dilakukan?



Saat menulis loop

- Tetapkan kondisi berhenti.
- Pastikan variabel bergerak ke arah kondisi itu.
- Uji kasus kecil dahulu.

Saat program macet

- Hentikan eksekusi.
- Catat nilai variabel pada beberapa iterasi.
- Periksa update variabel.

Tiga jenis kesalahan yang sering muncul

Syntax error

```
if x > 0  
    print(x)
```

Aturan penulisan Python tidak terpenuhi.

Runtime error

```
x = 3 / 0  
print(x)
```

Kode valid, tetapi gagal saat dijalankan.

Logical error

```
def rata(a):  
    return sum(a) / 2
```

Program berjalan, tetapi jawabannya salah.

Debugging dimulai dengan mengklasifikasi jenis kesalahan, bukan langsung mengubah banyak baris kode.

Runtime error dan exception

```
def rasio(a, b):  
    return a / b  
  
print(rasio(10, 2))  
print(rasio(10, 0))
```

Apa yang terjadi?

- Pemanggilan pertama berhasil.
- Pemanggilan kedua menimbulkan `ZeroDivisionError`.
- Program berhenti sebelum baris berikutnya sempat dijalankan.



Menangani exception tanpa menyembunyikan masalah

```
def rasio(a, b):  
    try:  
        return float(a) / float(b)  
    except ZeroDivisionError:  
        return None  
    except (TypeError, ValueError):  
        return None
```

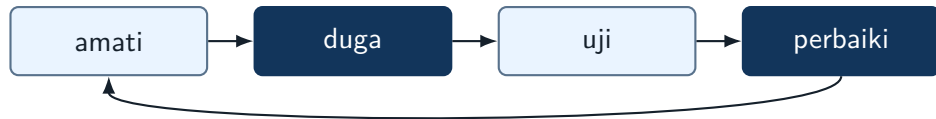
Kegunaan

- Program tetap terkendali.
- Kasus input tidak valid bisa ditangani.
- Fungsi memiliki perilaku yang bisa dijelaskan.

Catatan

- Jangan memakai except terlalu umum.
- Jangan menyembunyikan error yang perlu diperbaiki.
- Tetapkan makna None dengan jelas.

Debugging sebagai eksperimen kecil



Yang diamati

- Input yang membuat gagal.
- Output yang berbeda dari prediksi.
- Nilai variabel antar-langkah.

Yang diubah

- Satu bagian kecil dahulu.
- Uji kembali dengan kasus yang sama.
- Tambahkan kasus batas.

Contoh program salah: rata-rata

```
def rata_rata(data):  
    total = 0  
    for x in data:  
        total = total + x  
    return total / len(data)  
  
print(rata_rata([10, 20, 30, 40]))
```

Tebak dahulu

- Berapa outputnya?
- Apakah semua elemen diproses?
- Baris mana yang posisinya tidak tepat?

Ini logical error: tidak selalu memunculkan pesan error, tetapi hasilnya tidak sesuai tujuan.

Memperbaiki dengan menelusuri variabel

Tabel jejak

iterasi	x	total
awal	–	0
1	10	10
2	20	30
3	30	60
4	40	100

```
def rata_rata(data):  
    total = 0  
    for x in data:  
        total = total + x  
    return total / len(data)
```

- return berada setelah loop selesai.
- Jejak variabel membantu melihat alur, bukan hanya hasil akhir.

Testing dengan assert

```
def jarak_1d(x, y):  
    return abs(x - y)  
  
assert jarak_1d(3, 3) == 0  
assert jarak_1d(3, 7) == 4  
assert jarak_1d(-2, 5) == 7  
assert jarak_1d(5, -2) == jarak_1d(-2, 5)
```

Kasus uji

- Kasus mudah dihitung manual.
- Kasus batas.
- Sifat matematika yang harus berlaku.

Makna

assert bukan dekorasi. Ia menyatakan harapan yang harus benar jika fungsi ditulis dengan tepat.

Terminasi: kapan algoritma berhenti?

Istilah

Suatu program atau algoritma dikatakan *halt* untuk sebuah input jika eksekusinya berhenti setelah sejumlah langkah berhingga.

Loop

- Ada kondisi berhenti.
- Ada variabel yang diperbarui.
- Pembaruan bergerak ke kondisi berhenti.

Rekursi

- Ada base case.
- Ada recursive case.
- Argumen menuju base case.

Pertanyaan kuncinya: keadaan program bergerak ke mana?

Pola aman: batas iterasi

```
x = 1.0
maks_iterasi = 20

for k in range(maks_iterasi):
    x_baru = 0.5 * (x + 2/x)
    if abs(x_baru - x) < 1e-8:
        break
    x = x_baru
```

Mengapa ada batas?

Agar program tetap selesai walaupun kriteria berhenti tidak tercapai karena kesalahan numerik atau tebakan awal buruk.

Yang perlu dicatat

Nilai akhir, jumlah iterasi, dan apakah proses berhenti karena konvergen atau karena mencapai batas.

Masalah matematis: menghitung akar kuadrat

Tujuan

Mencari pendekatan untuk $\sqrt{n}$ berarti mencari x sehingga

$$x^2 - n = 0.$$

Newton-Raphson

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Untuk $f(x) = x^2 - n$

$$f'(x) = 2x$$

$$x_{k+1} = \frac{1}{2} \left(x_k + \frac{n}{x_k} \right).$$

- Ini contoh loop yang harus memiliki alasan berhenti.
- Perbaikan berhenti dapat memakai toleransi atau batas iterasi.

Program salah: iterasi Newton

```
def akar_newton(n):  
    x = 0  
    for k in range(20):  
        x = 0.5 * (x + n/x)  
    return x
```

Tebak dahulu

- Apa yang terjadi untuk $n = 2$?
- Baris mana yang menyebabkan masalah?
- Bagaimana memilih tebakan awal yang lebih aman?

Kesalahan bukan pada rumus Newton, tetapi pada keadaan awal yang membuat pembagian tidak terdefinisi.

Versi yang dapat diperiksa

```
def akar_newton(n, tol=1e-8, maks_iterasi=20):  
    if n < 0:  
        return None  
    if n == 0:  
        return 0.0  
    x = n / 2  
    for k in range(maks_iterasi):  
        x_baru = 0.5 * (x + n/x)  
        if abs(x_baru - x) < tol:  
            return x_baru  
        x = x_baru  
    return x
```

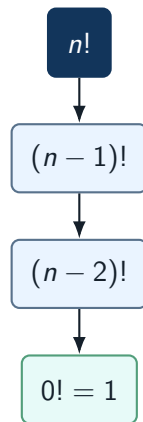
- Ada validasi domain.
- Ada tebakan awal.
- Ada kondisi berhenti dan batas iterasi.

Rekursi muncul dari masalah yang berulang

Contoh matematika

$$n! = n(n-1)!, \quad 0! = 1.$$

Masalah menghitung $n!$ memuat masalah yang lebih kecil, yaitu menghitung $(n-1)!$.



Rekursi bukan sekadar fungsi memanggil dirinya sendiri; rekursi memakai struktur masalah yang dapat diperkecil.

Implementasi rekursi: faktorial

```
def faktorial(n):  
    if n == 0:  
        return 1  
    return n * faktorial(n - 1)  
  
print(faktorial(5))
```

Dua bagian

- Base case: $n == 0$.
- Recursive case:
faktorial($n-1$).

Telusuri

$5! \rightarrow 4! \rightarrow 3! \rightarrow 2! \rightarrow 1! \rightarrow 0!$

Program salah: rekursi tanpa arah berhenti

```
def turun(n):  
    if n == 0:  
        return 0  
    return turun(n + 1)  
  
print(turun(3))
```

Tebak dahulu

- Apakah base case ada?
- Apakah pemanggilan berikutnya mendekati base case?
- Nilai n bergerak ke mana?

Base case saja tidak cukup. Recursive case harus membawa input menuju base case.

Masalah

Diberikan barisan

$$a_0 = 2, \quad a_n = 3a_{n-1} + 1.$$

Hitung a_1 , a_2 , dan a_3 .

Struktur rekursif

$$a_1 = 3a_0 + 1,$$

$$a_2 = 3a_1 + 1,$$

$$a_3 = 3a_2 + 1.$$

Setiap langkah membutuhkan nilai sebelumnya.

Implementasi barisan rekursif

```
def a(n):  
    if n == 0:  
        return 2  
    return 3*a(n - 1) + 1  
  
for k in range(4):  
    print(k, a(k))
```

Yang harus dijelaskan

- Mengapa $n == 0$ menjadi base case?
- Mengapa $n-1$ menuju base case?

Uji sederhana

Cocokkan output program dengan hitungan manual untuk a_0, a_1, a_2, a_3 .

Rekursi bisa benar tetapi mahal

```
def fib(n):  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
    return fib(n-1) + fib(n-2)
```

Pertanyaan

- Nilai mana yang dihitung berulang?
- Mengapa *fib(5)* masih aman, tetapi *fib(40)* terasa berat?
- Kapan versi iteratif lebih masuk akal?

Rekursi membantu mengekspresikan struktur masalah, tetapi tidak selalu menjadi implementasi paling efisien.

```
def jumlah_sampai(n):  
    total = 0  
    i = 1  
    while i <= n:  
        total = total + i  
    return total
```

Tugas kelas

- 1 Prediksi apakah program berhenti.
- 2 Sebutkan variabel yang seharusnya berubah.
- 3 Perbaiki satu baris saja.
- 4 Tambahkan satu assert.

```
def maksimum(data):  
    m = 0  
    for x in data:  
        if x > m:  
            m = x  
    return m  
  
print(maksimum([-5, -2, -9]))
```

Tugas kelas

- 1 Mengapa hasilnya salah?
- 2 Input seperti apa yang menyembunyikan kesalahan ini?
- 3 Perbaiki inisialisasi.
- 4 Uji dengan semua data negatif.

```
def pangkat(a, n):  
    if n == 0:  
        return 1  
    return a * pangkat(a, n)
```

Tugas kelas

- 1 Base case-nya apa?
- 2 Recursive case-nya apa?
- 3 Apakah argumen mendekati base case?
- 4 Tulis versi yang benar.

Pola kerja untuk menyelesaikan soal tantangan



Sebelum coding

- Tulis input dan output.
- Buat contoh kecil yang bisa dihitung manual.
- Putuskan apakah perlu loop atau rekursi.

Sesudah coding

- Uji normal, batas, dan kasus salah.
- Periksa terminasi.
- Tambahkan assert atau pesan error yang jelas.

Konsep yang dipakai

- Exception handling.
- Debugging syntax, runtime, logical error.
- Testing dengan assert.
- Halt atau terminasi.
- Rekursi dan tracing pemanggilan.

Target kerja

- Mampu menjelaskan mengapa kode salah.
- Mampu memperbaiki kode dengan alasan.
- Mampu membuat fungsi rekursif sederhana.
- Mampu membedakan benar, berhenti, dan efisien.

Materi quiz

- Membedakan syntax error, runtime error, dan logical error.
- Membaca kode dengan try/except.
- Menentukan output kode yang memakai return dan None.
- Menilai apakah loop atau rekursi berhenti.
- Menentukan base case dan recursive case.

Cara belajar

Prediksi output dahulu, baru jalankan Python. Jawaban yang baik memuat alasan, bukan hanya hasil akhir.

Yang dikumpulkan

- 1 Perbaiki tiga kode salah dari latihan kelas.
- 2 Fungsi `akar_newton` dengan validasi input dan batas iterasi.
- 3 Fungsi rekursif untuk satu barisan matematika.
- 4 Minimal lima `assert` untuk fungsi yang dibuat.
- 5 Catatan singkat: kasus mana yang paling mudah gagal dan bagaimana Anda mengetahuinya.

Boleh menggunakan Google Colab, Jupyter, Spyder, VS Code, atau IDE Python lain. Kode harus dapat dijalankan ulang.

Yang perlu dikuasai

- Program yang berhenti.
- Error yang dapat dibaca.
- Fungsi yang diuji.
- Rekursi yang memiliki arah berhenti.

Pertanyaan sebelum selesai

Untuk setiap program yang dibuat, tanyakan:

- 1 Apa input yang diizinkan?
- 2 Apa output yang dijanjikan?
- 3 Apa kasus batasnya?
- 4 Mengapa program berhenti?

- Bradley N. Miller dan David L. Ranum. *Problem Solving with Algorithms and Data Structures Using Python*, Release 3.0. Bab rekursi dan analisis algoritma.
- David J. Pine. *Introduction to Python for Science and Engineering*. Bagian fungsi, kontrol alur, NumPy, dan komputasi numerik.
- Wes McKinney. *Python for Data Analysis*, 3rd ed. Bagian Python dasar, fungsi, dan workflow berbasis notebook.

Rujukan digunakan sebagai dasar gaya materi: mulai dari problem, tulis algoritma, implementasikan kode, lalu uji hasil.